DEPI INDUSTRY GRADUATION PROJECT—DOCUMENTATION PROCESS.

3: Requirements Gathering & 4: System Analysis & Design.

Project Title: ***"Adaptive Learning Assistant: An AI-Powered Learning Chatbot for Dynamic Student Engagement."***

GIZ5_IND1_G1

Project Members:

1- Hosam Hasan Abdelhady Ibrahim. (Team Leader)

2- Omnia Reda Mohamed Rizk Ghalwash.

3- Mohamed Hussien Mohamed.

4- Doaa Abdelraouf Mahmoud Elganzoury.

SECTION 3: REQUIREMENTS GATHERING.

3.1 Stakeholder Analysis.

A comprehensive analysis to identify key stakeholders, their primary system objectives, and their specific technical or operational needs:

Stakeholder Role	Key Stakeholders	Primary Objectives	System Needs & Expectations
1- Students / Learners.	Undergraduate, postgraduate & technical trainees.	Access quick, accurate course explanations, generate self-assessment quizzes, and obtain visual learning aids.	Low-latency Telegram bot interface, precise answers derived strictly from course material, instant MCQ generation.
2- Academic Instructors.	Professors & course coordinators.	Automate student query handling, ensure academic integrity, and monitor curriculum coverage.	Simple document ingestion pipeline, strict domain guardrails preventing off-topic AI responses, query logging.

3- Evaluators/ Mentors.	DEPI Industry panel & reviewers.	Verify project compliance against technical, architectural, and educational standards.	Clear code documentation, reproducible n8n JSON workflows, robust test coverage, and complete GitHub submission.

3.2 User Stories & Use Cases.

- **US-01 (Textual RAG QA):** As a student, I want to ask natural language questions about my course syllabus via Telegram so that I can receive immediate, context-accurate explanations derived strictly from official textbook materials.
- **US-02 (MCQ Quiz Generation):** As a student, I want to request custom multiple-choice practice questions so that I can test my comprehension of specific topics with structured answers and immediate feedback.
- **US-03 (Visual Learning Aids):** As a student, I want to request charts or visual diagrams so that complex concepts can be rendered into intuitive QuickChart visualizations or generated concept images.
- **US-04 (Curriculum Ingestion):** As an instructor/admin, I want to upload new PDF textbooks into an automated pipeline so that the chatbot vector store updates seamlessly without manual programming.
- **US-05 (Guardrail Protection):** As an instructor, I want the system to reject file uploads and non-curriculum queries so that students do not abuse the system or receive hallucinated outside knowledge.

3.3 Functional Requirements.

- **FR-01 (Offline Ingestion - Document Fetching):** The system shall fetch PDF documents via HTTP URL or local binary loading using n8n file reading nodes.
- **FR-02 (Offline Ingestion - Vector Embedding & Indexing):** The system shall extract raw text from PDF files, split text into chunks using recursive character splitters, generate dense vector embeddings using Google Gemini (text-embedding-004), and insert vectors into Supabase Vector Store.
- **FR-03 (Offline Ingestion - Status Tracking):** The system shall write indexing metadata and complete status updates to an SQL database *(This step is not mandatory in the workflow)*.
- **FR-04 (Runtime - Telegram Listening):** The system shall capture incoming text messages, user IDs, and metadata in real time via a Telegram trigger node.
- **FR-05 (Runtime - File Guardrail Filtering):** The system shall inspect incoming Telegram payloads; if documents, photos, or videos exist (IF = True), it shall immediately transmit a rejection notice and stop execution.

- **FR-06 (Runtime - Intent Classification):** The system shall route clean text queries through a Google Gemini Chat classifier equipped with a Memory Tool and System Prompt Template to categorize intents into TEXT_QA, MCQ_GEN, IMAGE_GEN, or OUT_OF_SCOPE.
- **FR-07 (Runtime - RAG Text QA):** The system shall query Supabase Vector Store for top-k context matches and synthesize answers using the Google Gemini RAG Model before delivering text responses to Telegram.
- **FR-08 (Runtime - MCQ Generation):** The system shall route quiz requests to an AI Agent; these outputs, structured JSON, contain questions, options, correct choices, and explanations.
- **FR-09 (Runtime - Visualization Routing):** The system shall route visualization requests to a dedicated AI Agent capable of triggering the QuickChart API for data plots and Imagen/DALL-E, for example, for image generation.

(The test of multiple used LLMs is still running)

- **FR-10 (Runtime - Out-of-Scope Enforcement):** The system shall instantly route non-curriculum or off-topic queries to a static Telegram rejection node.

3.4 Non-Functional Requirements.

- **Performance & Speed:** End-to-end response time for RAG queries must remain ***under 5 seconds***.
- **Security & Guardrails:** ***100% interception rate*** for file attachments. Zero tolerance for unverified outside-domain knowledge hallucination.
- **Availability & Reliability:** The n8n webhook listener and runtime bot must maintain ***99.5% operational uptime*** with automatic error recovery in n8n execution nodes.
- **Usability & Accessibility:** Interface hosted entirely on Telegram requiring zero client software installation, providing structured formatting with Markdown and formatted JSON.
- **Scalability & Maintainability:** Modular architecture decoupling the ingestion pipeline from runtime agents, allowing independent scaling of Supabase vector storage and Gemini API quotas.

SECTION 4: SYSTEM ANALYSIS & DESIGN.

4.1 Problem Statement & Objectives.

Modern high/higher education environments present students with vast, static textbooks and slide decks that lack interactive engagement. Traditional generic AI chatbots often hallucinate, lack domain restriction, or accept unauthorized files that violate academic evaluation standards.

The Adaptive Learning Assistant addresses this with a dual-pipeline architecture built in n8n. By embedding course materials into Supabase Vector Store via Google Gemini text-embedding-004 and enforcing strict intent-classification guardrails, the system delivers verified, multi-modal pedagogical interaction (TEXT_QA, JSON MCQs, and QuickChart/Imagen diagrams) strictly within authorized course bounds.

4.2 Use Case Descriptions & Traceability Matrix.

Use Case ID & Name	Primary Actor	Pre-Conditions	Main Flow & Execution Path
UC-01: Curriculum Ingestion.	Course Admin / System.	PDF course document hosted at reachable HTTP URL or uploaded locally.	1. Trigger Manual Ingestion. 2. Fetch PDF & Load Binary. 3. Extract PDF Text. 4. Generate Gemini Embeddings. 5. Insert Vectors into Supabase. 6. Update SQL status log (optional).
UC-02: Student Query & RAG QA.	Student.	Telegram bot active; curriculum vectors loaded in Supabase.	1. Student sends text query via Telegram. 2. IF Guardrail checks for files (None found). 3. Intent Classifier categorizes query as TEXT_QA. 4. Supabase retrieves matching context. 5. Gemini Chat (for example) generates answer. 6. Bot sends text response to Telegram.
UC-03: Practice Quiz (MCQ) Request.	Student.	Telegram bot active; student requests practice questions.	1. Student asks for quiz on specific topic. 2. IF Guardrail checks for files (None found).

			3. Intent Classifier flags MCQ_GEN. 4. AI Agent generates structured JSON questions & explanations. 5. Bot sends formatted quiz to Telegram.
UC-04: Visual Aid / Diagram Request.	Student.	Telegram bot active; visual learning query submitted.	1. Student requests chart or visual diagram. 2. Intent Classifier flags IMAGE_GEN. 3. AI Visualization Agent invokes QuickChart API or Image. 4. Bot transmits generated image/photo to Telegram.
UC-05: Guardrail Interception.	Student / External.	Student sends file attachment or off-topic question.	1. If payload contains document/photo/video → IF Node triggers TRUE → Send Rejection & Stop Execution. 2. If text is off-topic → Intent Classifier flags OUT_OF_SCOPE → Send Out of Scope Rejection.

4.3 Software Architecture.

The system follows an **Event-driven micro-orchestration architecture** leveraging n8n as the workflow engine. The design cleanly decouples the system into two distinct operational pipelines:

```
+-----------------------------------------------------------------------------------------+
|                                  N8N WORKFLOW ENGINE                                    |
|                                                                                         |
|  [ WORKFLOW 1: OFFLINE CURRICULUM INGESTION PIPELINE ]                                  |
|  Manual Trigger -> HTTP Request -> Read Binary -> Extract PDF -> Gemini Embeddings -> Supabase  |
|                                                  |          |
```

```
|                                                    v          |
|                                                [ SQL Database ]  |
|                                                                  |
|  [ WORKFLOW 2: RUNTIME STUDENT CHAT ASSISTANT PIPELINE ]                                  |
|  Telegram Trigger -> IF Check (Files?) --(True)--> Telegram Rejection -> Stop Execution          |
|                    |                                             |
|                  (False)                                          |
|                    v                                             |
|             Google Gemini Classifier                                          |
|           (Chat Model + Memory + Prompt)                                        |
|                    |                                             |
|     +------------------+------------------+------------------+------------------+          |
|     | (TEXT_QA)        | (MCQ_GEN)        | (IMAGE_GEN)      | (OUT_OF_SCOPE)   |          |
|     v                  v                  v                  v                  |          |
|  Supabase Vector     AI Agent           AI Agent           Telegram Out of Scope   |          |
|  + Gemini RAG        (MCQ Gen)          (QuickChart/Imagen) Rejection Message      |          |
|     |                  |                  |                                |          |
|     v                  v                  v                                |          |
|  Telegram Send       Telegram Send      Telegram Send                                |          |
|  Text                MCQ Text           Photo/Image                                  |          |
+-------------------------------------------------------------------------------------------------+
```

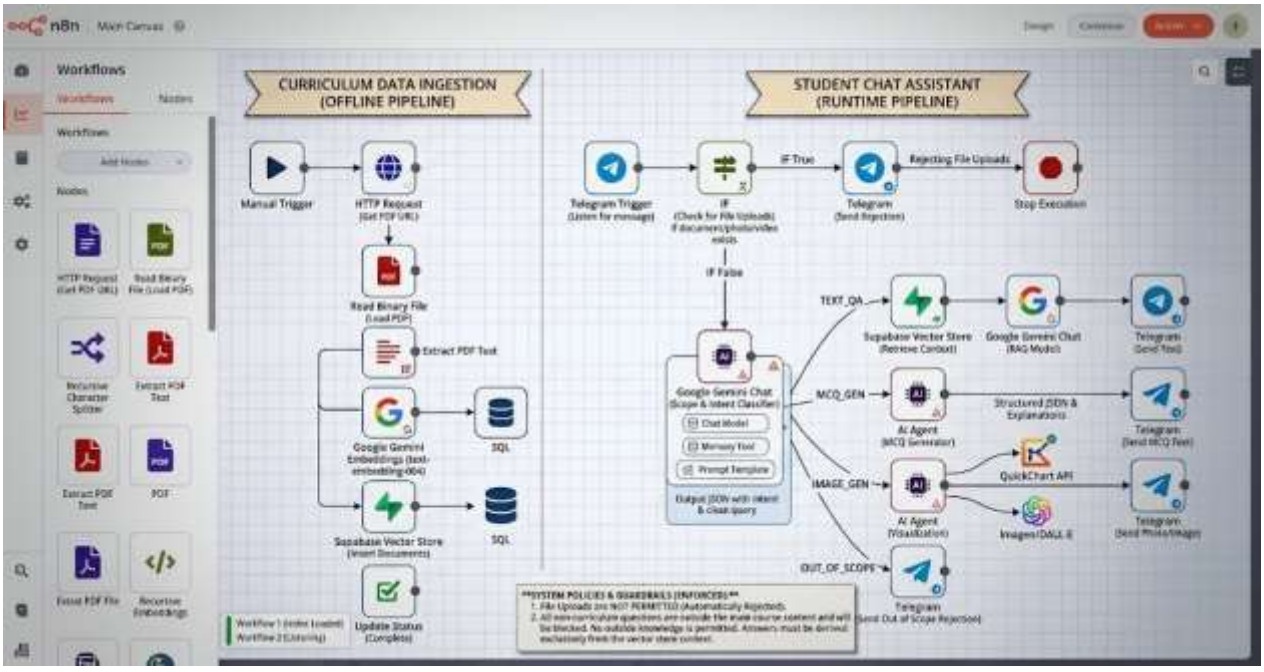


Figure 1. An n8n interactive workflow of the AI Adaptive Learning Assistant.

4.4 Database Design & Data Modeling.

The system data layer combines PostgreSQL relational tables for logging and audit trails with Supabase Vector (pgvector) extension for semantic similarity retrieval.

(Logical & Physical Schema Specification)

```
-- 1. Vector Store Schema (Supabase pgvector)
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE documents vector (
   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
   content TEXT NOT NULL,
   metadata JSONB,
   embedding VECTOR(768) -- Dimension matching Google Gemini text-embedding-004
);

-- Index for fast Cosine Distance Retrieval
CREATE INDEX ON documents_vector USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

-- 2. Ingestion Status Log Schema (SQL)
CREATE TABLE ingestion_status (
   job_id BIGSERIAL PRIMARY KEY,
   pdf_url VARCHAR(512) NOT NULL,
   status VARCHAR(50) NOT NULL, -- 'Pending', 'In Progress', 'Complete', 'Failed'
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Session Chat Memory Schema (SQL)
CREATE TABLE chat_memory (
   session_id VARCHAR(255) PRIMARY KEY, -- Telegram Chat ID / User ID
   messages JSONB NOT NULL,
   last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

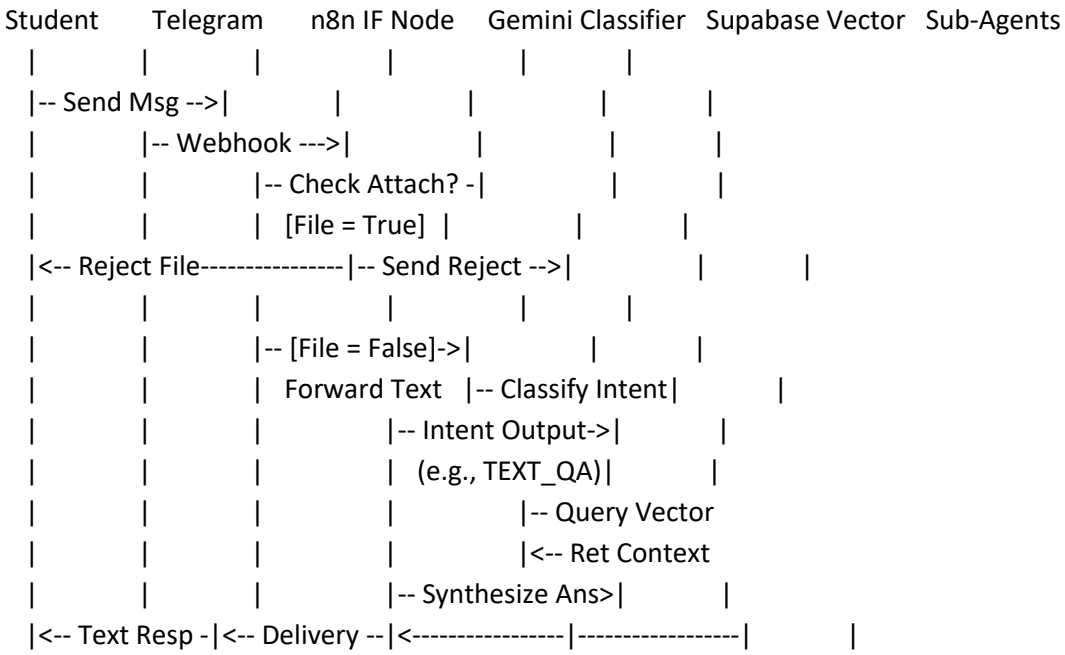
4.5 Data Flow & System Behavior Diagrams.

Data Flow Diagram (DFD Level 1 Textual Representation)

1. **External Entity (Student):** Sends a message or file via the Telegram Interface.

2. **Process 1.0 (File Policy Enforcement):** Evaluates if the message contains an attachment. If yes, transmits a rejection and stops. If no, forwards query text.
3. **Process 2.0 (Intent & Scope Classifier):** Analyzes query text against system prompt and session memory; assigns intent category (TEXT_QA, MCQ_GEN, IMAGE_GEN, OUT_OF_SCOPE).
4. **Process 3.0 (Context Retrieval & RAG):** For TEXT_QA, searches documents vector table in Supabase via Cosine Similarity; passes context + query to Gemini Chat.
5. **Process 4.0 (Sub-Agent Dispatching):** Executes a specific agent (MCQ JSON Generator, QuickChart Plotter, or Imagen API).
6. **Data Store (Supabase / SQL):** Receives vector embeddings during ingestion and serves context vectors during runtime.

Sequence Diagram Representation (Runtime Flow)



4.6 UI/UX Design & Prototyping Guidelines.

The UI/UX design of the **Adaptive Learning Assistant** focuses on providing an intuitive, interactive, and personalized learning experience through a **Telegram**-based chatbot interface.

The design emphasizes simplicity, accessibility, adaptive learning, and continuous student engagement through the following guidelines:

- **Conversational Interface and Minimalism:** The system uses **Telegram** as its primary user interface (**see Figure 2**), allowing students to interact with the AI-powered learning assistant through a familiar messaging environment without requiring a separate application. A clean, organized interface provides easy access to key features, including asking questions, taking quizzes, and accessing study resources,
- **Interactive Learning and Instant Feedback:** The chatbot supports interactive quizzes and adaptive practice activities to reinforce students' understanding. Multiple-choice questions present clearly labeled answer options (A, B, C, and D), followed by immediate feedback, correct answers, and explanatory responses. Question difficulty can adapt to student performance, promoting progressive learning and continuous improvement.
- **Visual Learning and Multimedia Integration:** The interface incorporates visual learning materials, including diagrams, charts, illustrations, and educational videos, accompanied by concise explanatory text. These visual elements help simplify complex concepts, improve comprehension, and support different learning preferences. Study resources, such as summary notes, video lectures, interactive diagrams, and practice questions, are organized for convenient access.
- **Personalized Progress Tracking:** The system provides a dedicated progress-tracking interface that displays overall learning progress, topic-specific performance, and quiz results. Visual indicators, such as progress bars and percentage scores, help students identify strengths, recognize areas for improvement, and track their learning development over time.
- **Clear Navigation and Consistent Visual Design:** The prototype maintains a consistent layout, color scheme, typography, and button style across its interfaces. Clearly labeled actions, such as *Ask a Question*, *Take a Quiz*, and *Study Resources*, enable students to navigate the system efficiently. Welcome screens, structured menus, and contextual action buttons further improve usability and reduce interaction complexity.
- **Notifications and Learning Reminders:** The chatbot provides timely reminders about pending quizzes, upcoming learning activities, and suggested topics for revision. Personalized notifications and encouraging messages help students maintain consistent study habits, remain motivated, and engage continuously with the learning process.
- **User-Friendly Guidance and Guardrail Messaging:** The system communicates limitations and restrictions through clear, informative, and non-punitive messages. For example, when a request is unsupported or outside the intended course scope, the chatbot explains the limitation and guides the student toward appropriate learning activities. This approach maintains a supportive educational environment while keeping interactions relevant to the project's learning objectives.

Overall, these UI/UX guidelines aim to create an accessible, engaging, and adaptive learning environment in which students can ask questions, practice concepts, explore multimedia resources, and monitor their progress through a unified conversational interface. The prototype reflects the project's primary objective of enhancing **dynamic student engagement through AI-powered personalized learning and continuous feedback**.



Figure 2: UI/UX Design and Prototype of the Adaptive Learning Assistant: An AI-Powered Learning Chatbot for Dynamic Student Engagement.

4.7 System Deployment & Technology Stack.

Layer / Domain	Selected Technology / API	Technical Role & Component Mapping
Workflow Orchestration Engine.	n8n (Self-Hosted / Cloud).	Central execution environment managing node workflows, webhooks, triggers, IF conditions, sub-agents, and branching logic.
Front-End User Interface.	Telegram Bot API.	Client interface capturing incoming student text messages and delivering multi-modal responses (text, JSON MCQs, images).

Vector Store & Database.	Supabase (pgvector) + PostgreSQL (optional).	Stores document vector embeddings (768-dim) for RAG context retrieval; stores execution logs and status metadata.
Large Language Model Engine.	Google Gemini API/ Claude (Chat & Embeddings).	Gemini or Claude text-embedding-004 for vector embeddings; Gemini Chat Model or Claude for intent classification and RAG response synthesis.
Visualization & Image APIs.	QuickChart API/ Gemini/ Imagen / DALL-E *(these examples are still under test).*	QuickChart API generates dynamic data charts; Gemeni/Imagen/DALL-E generates visual concept diagrams for the visualization sub-agent.
Version Control & Deployment	GitHub Repository.	Stores exported n8n workflow JSON files, SQL schema definitions (if existed), test scripts, and complete documentation of the project.

4.8 System Policies & Enforced Guardrails.

As strictly specified in the n8n main canvas architecture, the system enforces mandatory security policies and guardrails at the runtime level:

SYSTEM POLICIES & GUARDRAILS (ENFORCED):

1. FILE UPLOAD POLICY:
File uploads (documents, photos, videos) are strictly NOT PERMITTED during runtime chat sessions. Any uploaded media triggers an automatic IF-node interception, sends a rejection message, and halts execution immediately via the 'Stop Execution' node.

2. DOMAIN BOUNDARY POLICY:
All non-curriculum questions are classified as outside the main course content and will be blocked. No outside knowledge is permitted; answers must be derived exclusively from the Supabase vector store context.